

PRINCÍPIOS DA LINGUAGEM C

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

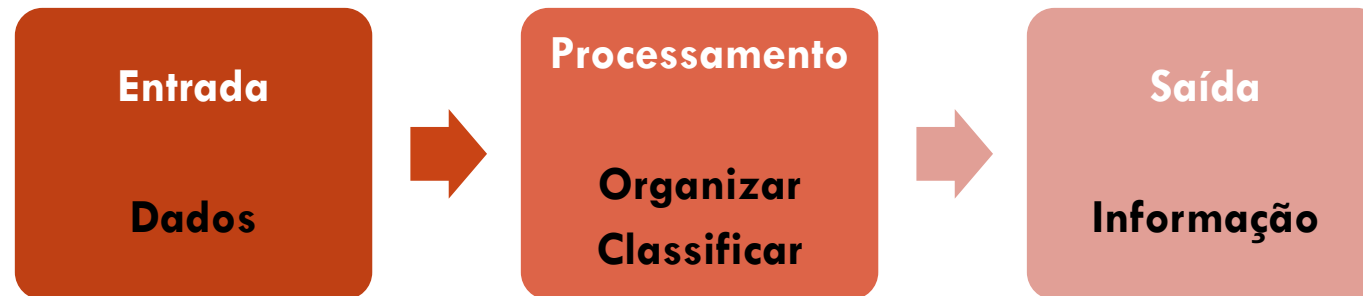
AGENDA

- Introdução
- Estrutura de um programa em C
- Constantes e variáveis
- Tipos de Dados
- Funções de entrada e saída
- Operadores
- Dúvidas



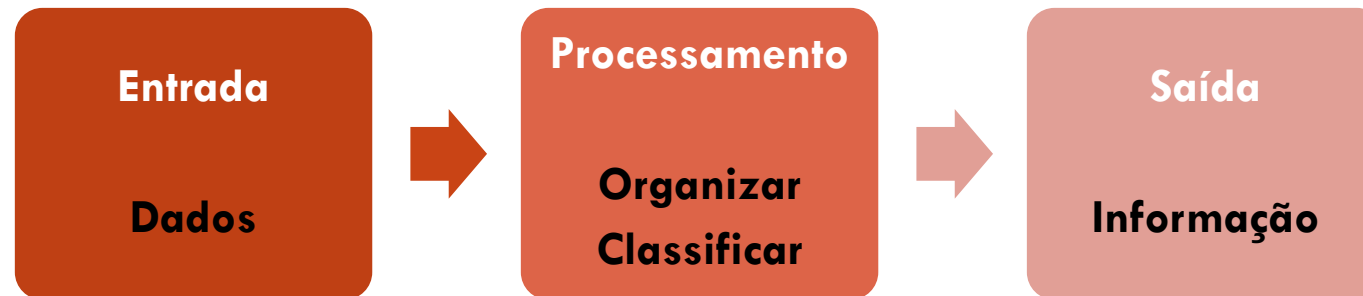
INTRODUÇÃO

- **Dados** são códigos que constituem a matéria prima da informação, ou seja, é a informação não tratada.
- Os dados representam um ou mais significados que isoladamente não podem transmitir uma mensagem ou representar algum conhecimento.
- Dados são sucessões de fatos brutos (números ou caracteres).



INTRODUÇÃO

- **Informações** são dados tratados. O resultado do processamento de dados são as informações.
- As informações tem significado, podem ser tomadas decisões ou fazer afirmações considerando as informações.
- O **conhecimento** vai além de informações, pois ele além de ter um significado tem uma aplicação.



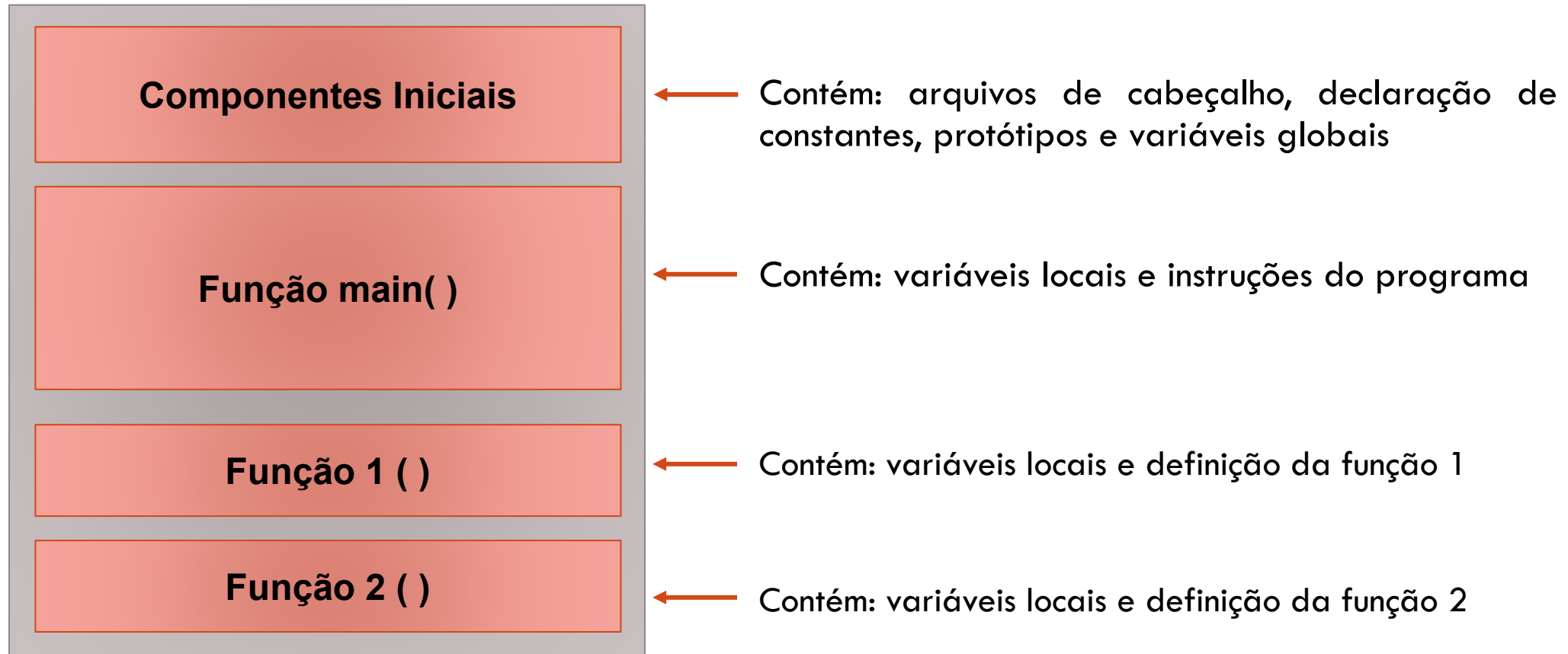
ESTRUTURA DE UM PROGRAMA EM C

- A linguagem C diferencia caracteres maiúsculas de caracteres minúsculos.
- Exemplo:
 - `variavel`
 - `Variavel`
 - `VARIABEL`
- Cuidado com os nomes de variáveis, funções, constantes (identificadores).
- Atenção: Todos os comandos em C são escritos com letras minúsculas.
- Apenas funções definidas por programadores podem ser descritas com letras maiúsculas.

ESTRUTURA DE UM PROGRAMA EM C

- Os programas são organizados da seguinte forma:
- `#include` : utilizado para inclusão de bibliotecas
- `#define` : utilizado para definição de constantes
- Variáveis globais : as quais são visíveis em todo o programa
- Protótipos de função: para declaração antes do seu uso
- `main()` : ponto de entrada do programa
- Demais funções : para modularização do código
- Comentários `//` ou `/*`: com objetivo de documentar o programa

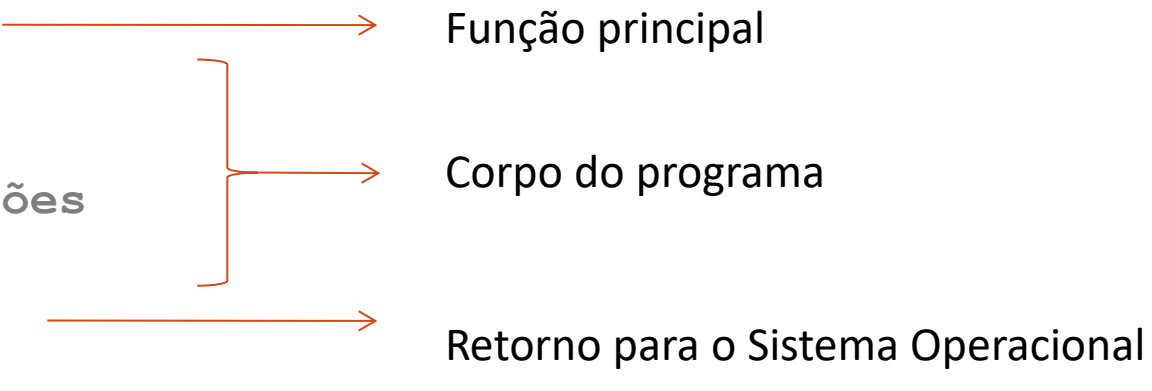
ESTRUTURA DE UM PROGRAMA EM C



ESTRUTURA DE UM PROGRAMA EM C

- O primeiro programa:

```
int main()
{
    //instruções
    return 0;
}
```



The diagram illustrates the structure of a C program with the following annotations:

- `int main()` is annotated with "Função principal" (Main function).
- The block between `{` and `}` is annotated with "Corpo do programa" (Program body).
- `return 0;` is annotated with "Retorno para o Sistema Operacional" (Return to the Operating System).

ESTRUTURA DE UM PROGRAMA EM C

```
#include <stdio.h> // Bibliotecas
#define PI 3.14159 // Constantes
int soma(int a, int b); // Protótipo de função
```

Componentes Iniciais

```
int main() {
    int resultado; // Variáveis
    resultado = soma(2, 3); // Chamada da função
    printf("Resultado: %d - Valor Constante: %f\n", resultado, PI);
    return 0;
}
```

Função Principal

```
int soma(int a, int b) // Definição da função
{
    return a + b; // Corpo da função
}
```

Função Soma

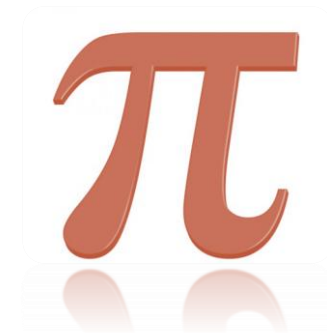
BIBLIOTECAS

- **Bibliotecas** contêm funções pré-definidas, utilizadas nos programas.
- A diretiva `#include` permite incluir uma biblioteca. Ela é seguida do nome da biblioteca entre sinais de maior (<) e menor (>).
- Exemplos:

Diretiva + Biblioteca	Descrição
<code>#include <stdio.h></code>	Funções de entrada e saída
<code>#include <stdlib.h></code>	Funções tipo padrão
<code>#include <math.h></code>	Funções matemáticas
<code>#include <conio.h></code>	Funções manipular caracteres na tela
<code>#include <string.h></code>	Funções de texto

CONSTANTES

- **Constantes** representam localizações de memória onde são armazenados dados que não podem ser modificados durante a execução do programa.
- **Nome de constantes**
 - Por convenção, constantes são criadas com letras maiúsculas
- **Declaração e inicialização de constantes**
 - Sintaxe: `#define NOME_CONST valor`
 - Exemplos: `#define PI 3.1415`
`#define MAX 1000`



VARIÁVEIS

- **Variáveis** são espaços de memória reservados que guardam valores durante a execução de um programa.
- Todas as variáveis em C devem ser declaradas, antes de serem usadas.
- O tipo de uma variável informa a quantidade de memória, em bytes, que a variável ocupará e a forma como um valor deverá ser armazenado.



VARIÁVEIS

- **Exercício:** Verifique se os nomes das variáveis abaixo são válidos:
 - a) total
 - b) 1total
 - c) char
 - d) _vaLOR
 - e) Valor1
 - f) ivalor
 - g) float



VARIÁVEIS

- Abaixo as regras para os **nomes** de identificadores:
- Só podem conter letras, números e `_` (*underline*).
- Não podem começar com número.
- Diferenciam maiúsculas de minúsculas (*case sensitive*).

- Não podem usar palavras reservadas:

<i>auto</i>	<i>double</i>	<i>int</i>	<i>struct</i>
<i>break</i>	<i>else</i>	<i>long</i>	<i>switch</i>
<i>case</i>	<i>enum</i>	<i>register</i>	<i>typedef</i>
<i>char</i>	<i>extern</i>	<i>return</i>	<i>union</i>
<i>const</i>	<i>float</i>	<i>short</i>	<i>unsigned</i>
<i>continue</i>	<i>for</i>	<i>signed</i>	<i>void</i>
<i>default</i>	<i>goto</i>	<i>sizeof</i>	<i>volatile</i>
<i>do</i>	<i>if</i>	<i>static</i>	<i>while</i>

- Não ultrapassar 32 caracteres.

VARIÁVEIS

- **Declaração** de variáveis
- Sintaxe: tipo nome_var;
- Exemplos: int x1, x2;
 float y, z, salario_mínimo;
 char letra;

- **Inicialização** de variáveis
- Sintaxe: nome_var = valor;
- Exemplos: x = 10;
 y = z = 10.5;
- **Declaração e inicialização** de variáveis
- Sintaxe: tipo nome_var = valor;
- Exemplo: int x = 10;

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>	<i>Máx</i>	<i>Obs</i>
<code>char</code>	1	-128	127	Tipo inteiro de 1 byte com sinal
<code>unsigned char</code>	1	0	255	Tipo inteiro de 1 byte sem sinal
<code>short int</code>	2	-32.768	32.767	Tipo inteiro de 2 bytes com sinal
<code>unsigned short int</code>	2	0	65.535	Tipo inteiro de 2 bytes sem sinal
<code>long int</code>	4	-2.147.483.648	2.147.483.647	Tipo inteiro de 4 bytes com sinal
<code>unsigned long int</code>	4	0	4.294.967.295	Tipo inteiro de 4 bytes sem sinal
<code>long long int</code>	8	-9.223.372.036. 854.775.808	9.223.372.036. 854.775.807	Tipo inteiro de 8 bytes com sinal
<code>unsigned long long int</code>	8	0	18.446.744.073 .709.551.615	Tipo inteiro de 8 bytes sem sinal

Tipos com *unsigned* (sem sinal) utilizam todos os seus bytes para compor o seu valor

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>	<i>Máx</i>	<i>Obs</i>
char	1	-128	127	Tipo inteiro de 1 byte com sinal
unsigned char	1	0	255	Tipo inteiro de 1 byte sem sinal

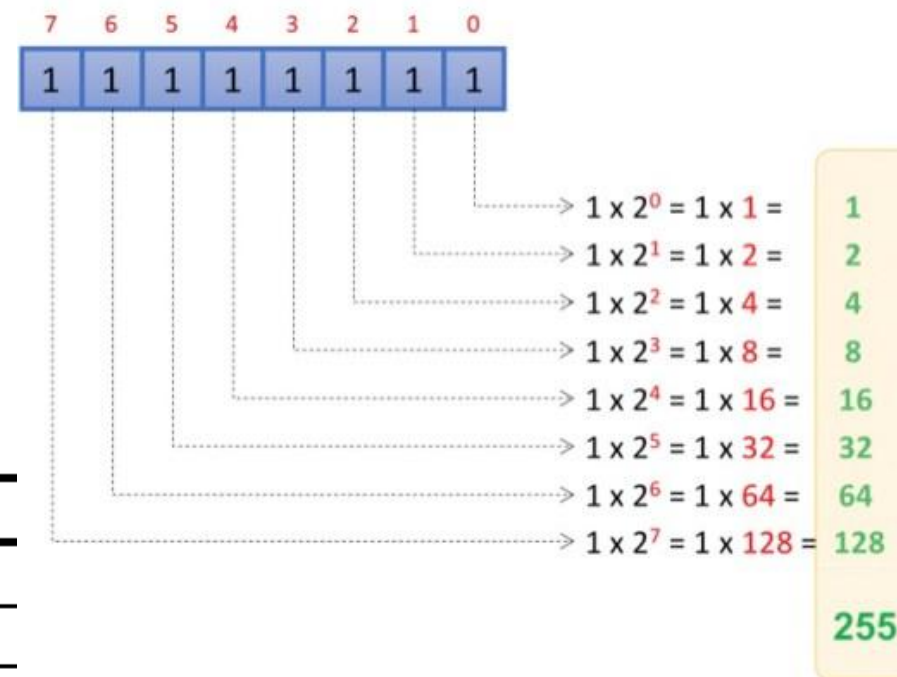
Decimal (interpretado)	Binário (8 bits)	char (signed)	unsigned char
0	00000000	0	0
1	00000001	1	1
127	01111111	127	127
-1	11111111	-1	255
-128	10000000	-128	128
-2	11111110	-2	254

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>
char	1	-128
unsigned char	1	0

Decimal (interpretado)	Binário (8 bits)	char (signed)	unsigned char
0	00000000	0	0
1	00000001	1	1
127	01111111	127	127
-1	11111111	-1	255
-128	10000000	-128	128
-2	11111110	-2	254



Complemento de Dois para -1 em 8 bits

Descrição	Binário (8 bits)
Valor positivo +1	00000001
Inverter todos os bits (complemento de um)	11111110
Somar 1 ao resultado	11111111

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>	<i>Máx</i>	<i>Obs</i>
char	1	-128	127	Tipo inteiro de 1 byte com sinal
unsigned char	1	0	255	Tipo inteiro de 1 byte sem sinal

Decimal (interpretado)	Binário (8 bits)	char (signed)	unsigned char
0	00000000	0	0
1	00000001	1	1
127	01111111	127	127
-1	11111111	-1	255
-128	10000000	-128	128
-2	11111110	-2	254

Exemplo de código:

```
char c = -1; //pode ser negativo
unsigned char uc = -1; //não pode ser negativo

printf("char c = %d\n", c);
printf("unsigned char uc = %u\n", uc);
```

```
char c = -1
unsigned char uc = 255
```

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>	<i>Máx</i>	<i>Obs</i>
char	1	-128	127	Tipo inteiro de 1 byte com sinal
unsigned char	1	0	255	Tipo inteiro de 1 byte sem sinal

Decimal (interpretado)	Binário (8 bits)	char (signed)	unsigned char
0	00000000	0	0
1	00000001	1	1
127	01111111	127	127
-1	11111111	-1	255
-128	10000000	-128	128
-2	11111110	-2	254

Exemplo de **overflow** no char sem sinal:

<i>Operação</i>	<i>Valor decimal</i>	<i>Valor binário</i>
Inicial	254	11111110
+1	255	11111111
+1 (overflow)	0	00000000
+1	1	00000001

254 – 255 – 0 (overflow volta ao menor valor) – 1 – 2 ...

TIPOS DE DADOS

- Tipo dados numéricos primitivos **inteiros** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Mín</i>	<i>Máx</i>	<i>Obs</i>
char	1	-128	127	Tipo inteiro de 1 byte com sinal
unsigned char	1	0	255	Tipo inteiro de 1 byte sem sinal

Decimal (interpretado)	Binário (8 bits)	char (signed)	unsigned char
0	00000000	0	0
1	00000001	1	1
127	01111111	127	127
-1	11111111	-1	255
-128	10000000	-128	128
-2	11111110	-2	254

Exemplo de **underflow** no char com sinal:

<i>Operação</i>	<i>Valor decimal</i>	<i>Valor binário</i>
Inicial	1	00000001
-1	0	00000000
-1 (underflow)	255	11111111
-1	254	11111110

1 – 0 – 255 (underflow volta ao maior valor) – 255 – 254 ...

TIPOS DE DADOS

- Tipo dados numéricos primitivos **reais** em C (32 bits)

<i>Tipo</i>	<i>Bytes</i>	<i>Faixa</i>	<i>Obs</i>
float	4	$\pm 10^{-38}$ a $\pm 10^{38}$	Tipo real de precisão simples
double	8	$\pm 10^{-308}$ a $\pm 10^{308}$	Tipo real de precisão dupla
long double	12	$\pm 10^{-4932}$ a $\pm 10^{4932}$	<i>Não suportado em alguns compiladores</i>

- float (precisão simples, ~7 casas decimais)
- double (precisão dupla, ~15 casas decimais)
- long double (maior precisão, nem sempre suportado)
- Faixa ampla de valores representáveis

TIPOS DE DADOS

- Existem **problemas de precisão** também, pois a representação binária de números decimais gera perdas
- Erros acumulados em operações repetidas
- Exemplo prático: subtração repetida de 0.1

```
#include <stdio.h>

int main(void)
{   float valor = 1000.0;
    int cont = 0;

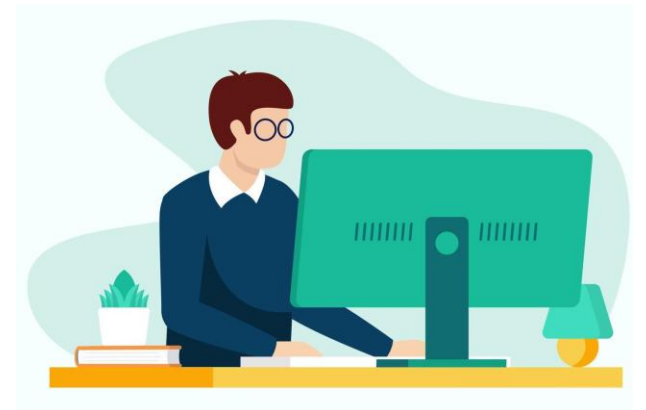
    while (cont < 1000 * 10)
    {   valor = valor - 0.1;
        cont = cont + 1;
    }

    printf("\nValor: %f  %d",
           valor, cont);

    return 0;
}
```

TIPOS DE DADOS

- Os **tipos primitivos** são a base de qualquer programa;
- Conhecer faixas e limites evita erros sutis;
- *Overflow*, *underflow* e precisão são problemas práticos reais



FUNÇÃO PRINTF()

- A função **printf()** é utilizada para exibir os dados na tela.
- Sintaxe: **printf(a1, a2, a3, ..., an);**
- Primeiro argumento (a1) chama-se string de formato, deve estar entre aspas duplas e pode conter:
 - Texto com ou sem códigos de barra invertida: **printf("Bom dia!");** ou **printf("Bom\ndia!");**
 - Especificadores de formato: **printf("O valor de x é %i",x);**
- Demais argumentos (a2, ..., an) chamam-se itens de dados. São quantos forem os especificadores de formato do primeiro argumento
 - **printf("FLOAT = %f INT = %i CHAR = %c", var1, var2, var3);**
 - **printf("Resultado = %.2f ", num);**

CÓDIGOS DE FORMATAÇÃO

- Também chamados de especificadores de formato são utilizados para especificar o tipo de dado que será lido ou escrito na tela.

Código	Tipo de dados
%c	char (caracteres individuais)
%s	char (conjunto de caracteres)
%d ou %i	int
%f	float
%lf	double
%e	Número em notação científica, por exemplo, 10.3e-8
%x	Número em hexadecimal, por exemplo, 32F2
%p	Endereço de memória (ponteiro)
%%	Imprime um %

CÓDIGOS DE BARRA INVERTIDA

- Muitas vezes é necessário inserir código de barra invertida para formatar a saída do texto.

Código	Formato
\n	Nova linha.
\t	Tabulação.
\a	Beep – Toque do auto-falante.
\\	Barra invertida.
\0	Zero.
\'	Aspas simples (apóstrofo).
\"	Aspas duplas.
\xdd	Representação hexadecimal.
\ddd	Representação octal.

FUNÇÃO SCANF()

- A função **scanf()** é utilizada para ler dados do teclado e colocar os valores fornecidos pelo usuário nas variáveis utilizadas como parâmetros da função.
- Sintaxe: **scanf(a1, a2, a3, ..., an);**
 - Primeiro argumento (a1) chama-se string de formato, deve estar entre aspas duplas e pode conter:
 - Especificadores de formato: **scanf("%i", &x);**
- Demais argumentos (a2,...,an) chamam-se itens de dados. São quantos forem os especificadores de formato do primeiro argumento e devem ser precedidos pelo operador & (endereço de)
 - **scanf("%f %i %c", &var1, &var2, &var3);**

OPERADORES

- A linguagem C é muito rica em operadores internos.
- As classes de operadores são:
 - Aritméticos;
 - Relacionais;
 - Lógicos.



OPERADORES

- O símbolo de igual (=) representa a **atribuição** de um valor à uma variável, onde a variável e o valor atribuído devem ser de tipos compatíveis.
- Sintaxe: `nome_var = expressão;`
- Exemplos: `x = 2;`
`y = x + 3;`
- **Conversão de tipos em atribuições**
- **Regra** : o valor do lado direito de uma atribuição é convertido para o tipo do lado esquerdo
- Exemplo: `int x;`
`float y;`
`x = y = 3.5;`

OPERADORES

- **Binários** - atuam sobre dois operandos:

Principais Operadores Aritméticos da Linguagem C		
Operação	Operador	Exemplo
Adição	+	$5 + 3 \rightarrow 8$
Subtração	-	$9 - 2 \rightarrow 7$
Multiplicação	*	$2 * 4 \rightarrow 8$
Divisão	/	$6 / 3 \rightarrow 2$
Módulo (resto da divisão inteira)	%	$5 \% 3 \rightarrow 2$

OPERADORES

- **Binários** - esses operadores possuem uma ordem de precedência:

Ordem dos Operadores	
Ordem	Operador
1º	Parênteses mais internos
2º	Potenciação e raiz
3º	* e /
4º	+ e -

OPERADORES

- Funções predefinidas para **double**
- Biblioteca - **math.h**:
- `atan(x)`: retorna arco tangente de `x`.
- `cos(x)`: retorna cosseno de `x`.
- `sin(x)`: retorna seno de `x`.
- `exp(x)`: retorna função exponencial natural (e^x).
- `pow(x,y)`: eleva `x` à potência `y`.
- `sqrt(x)`: retorna a raiz quadrada de `x`.
- `log(x)`: retorna o logaritmo natural de `x`.

- Funções predefinidas para **inteiros**
- Biblioteca - **stdlib.h**:
- `abs(x)`: retorna o valor absoluto de `x`.
- `atoi(str)`: converte string em int.
- `atol(str)`: converte string em long int.
- `atoll(str)`: converte string em long long int.

OPERADORES

- **Unários** – atuam sobre um operando:

Operadores Aritméticos Unários da Linguagem C		
Operação	Operador	Exemplo
Menos Unário	-	$-x \rightarrow x * (-1)$ // Multiplica a variável por -1
Incremento	++	$++x \rightarrow x = x + 1$ // Incrementa variável antes de usá-la $x++ \rightarrow x = x + 1$ // Incrementa variável depois de usá-la
Decremento	--	$--x \rightarrow x = x - 1$ // Decrementa variável antes de usá-la $x-- \rightarrow x = x - 1$ // Decrementa variável depois de usá-la

OPERADORES

- **Exercício:** Qual o valor das variáveis x , y e z depois da seguinte sequência de operações?

```
int x, y, z;
```

```
x = y = 10;
```

```
z = (x++);
```

```
x = -x;
```

```
y++;
```

```
x = x + y - (z--);
```



OPERADORES

- **Relacionais** - comparam expressões e resultam em verdadeiro ou falso:

Operadores de Igualdade			
Operador	Símbolo	Exemplo	Resultado
Igualdade	==	10 == 20	Falso
Diferença	!=	10 != 20	Verdadeiro
Operadores Relacionais			
Operador	Símbolo	Exemplo	Resultado
Maior que	>	10 > 20	Falso
Menor que	<	10 < 20	Verdadeiro
Maior ou igual que	>=	5 >= 5	Verdadeiro
Menor ou igual que	<=	12 <= 20	Verdadeiro

OPERADORES

- **Operadores lógicos** são usados para relacionar duas expressões ou apenas uma que é o caso do operador de negação.
- Operador **AND** (&&)
- Resulta em verdadeiro se ambas expressões forem verdadeiras
- Operador **OR** (||)
- Resulta em verdadeiro se pelo menos uma expressão for verdadeira
- Operador **NOT** (!)
- Resulta em verdadeiro se a expressão for falsa

OPERADORES

- **Lógicos** - utilizados entre expressões com operadores relacionais:

Operadores Lógicos			
Operador	Representação na Lógica Matemática	Representação na Linguagem C	Exemplo
Conjunção - AND	E ($\wedge$)	<code>&&</code>	<code>A && B</code>
Disjunção - OR	OU ($\vee$)	<code> </code>	<code>A B</code>
Negação - NOT	Negação ($\sim$)	<code>!</code>	<code>!A</code>

OPERADORES

- **Lógicos** - utilizados entre expressões com operadores relacionais:

Operadores Lógicos			
Operador	Representação na Lógica Matemática	Representação na Linguagem C	Exemplo
Conjunção - AND	E ($\wedge$)	&&	A && B
Disjunção - OR	OU ($\vee$)		A B
Negação - NOT	Negação ($\sim$)	!	!A

OPERADORES

- **Exercício:** Qual o retorno de cada expressão abaixo?

• <code>1 == 1 && 2 == 2</code>	• retorna verdadeiro
• <code>2 > 1 && 2 != 2</code>	• retorna falso
• <code>1 != 1 2 != 2</code>	• retorna falso
• <code>2 >= 1 2 != 2</code>	• retorna verdadeiro
• <code>!(2 > 2)</code>	• retorna verdadeiro
• <code>!(2 == 2)</code>	• retorna falso



DÚVIDAS

- Questionamentos?

